SINEX TRF Studio 1.5.2: User Manual

Gerasimos M. Dossas, Dimitrios Ampatzidis, Dimitrios Botsis

Corresponding author: D. Ampatzidis, dampatzi@ihu.gr

October 2026

Table of Contents

1 Introduction 2
1.1 About SINEX TRF Studio 2
1.2 Release, licence and citation 2
2 Requirements and installation 2
2.1 Requirements 2
2.2 Installing 3
2.3 Starting the program 4
3 Quick start with the sample data 4
3.1 Sample files 4
3.2 Walkthrough 5
4 Expected results 5
5 Program reference 7
5.1 Status widget and variance factor 7
5.2 Benchmark recording 8
5.3 Covariance Matrix tab 8
5.4 Datum Effect tab 11
5.5 Stations tab 13
5.6 Block Export tab 14
5.7 Settings menu 15
5.8 Binary file library 16
5.9 Information tab 17
6 Command line 18
7 Output files 18
8 Reproducing the published runs 19
8.1 ITRF2014 GNSS (control run) 19
8.2 ITRF2020-u2023 GNSS 22
Appendix A: Background, mathematical modelling and equations 23
The SINEX File Format 23
The Datum Effect Formulas 25

The Definition of an Episode 26
Appendix B: Architecture and parsing engine 27
Architecture and workflow 27
Parsing engine 28
Appendix C: Version history 29
References 29

1 Introduction

This manual accompanies the GPS Toolbox paper on SINEX TRF Studio. It describes how to install and start the software, the sample data set with the results it should produce, every tab of the window, and, in the appendices, the background and equations behind its computations. It also documents the ITRF2014 and ITRF2020-u2023 runs reported in the paper, so that they can be repeated.

1.1 About SINEX TRF Studio

SINEX TRF Studio is an operating-system-agnostic application in Python 3 with a PyQt6 interface. It parses the SINEX blocks needed for covariance-based frame analysis and computes the normal equation matrix with constraint removal, the covariance matrix of the frame parameters (Σ_θ), the correlation matrix R, and the formal uncertainties of the fourteen datum parameters. An episode and uncertainty limit filter, fed by two sigma sources and a manual override, thins the parameter set before the datum computation. The software is released under GPLv3 as version 1.5.2 (Dossas and Ampatzidis, 2026).

Furthermore, the SINEX TRF Studio package allows handling international (or otherwise large) SINEX files. An optimised engine loads in memory each file and parses the covariance block with parallel workers. To work around large file sizes, the operations of the software suite are done in a memory-conscious manner, and each loaded file is kept as a binary copy that later loads open in a fraction of a second, with the matrix read from disk as it is needed.

1.2 Release, licence and citation

SINEX TRF Studio 1.5.2 has the GPLv3 licence, which it inherits from PyQt6. The source code, the issue tracker and the releases are on GitHub (https://github.com/gerxd/SINEX_TRF_Studio). Zenodo archives every release. The DOI https://doi.org/10.5281/zenodo.22548394 always resolves to the latest version (Dossas and Ampatzidis, 2026). The file CITATION.cff in the repository root gives the citation data.

The version history is given in Appendix C.

2 Requirements and installation

2.1 Requirements

SINEX TRF Studio is an operating system (OS) agnostic suite that can operate in Windows, macOS and Linux. It consists of several files in its main directory and utilises a variety of external libraries to achieve all functionality. The construction of this software suite utilised the Python3 programming language. An

OS-specific executable is included in the repository, that handles the installation of the dependencies into a virtual environment on first launch and the program execution on second.

Several third-party libraries were used in conjunction to make this software. PyQt6 (Riverbank Computing, 2026) was used for all graphical user interface (GUI) use cases. The mathematics engine was done solely with the NumPy (Harris et al., 2020) package that incorporates the OpenBLAS implementation. The numerical stability of all matrix operations therefore is done on established and widely validated libraries. The plots use matplotlib (Hunter, 2007), seaborn and PyQtGraph. pandas, openpyxl, csv, were used to cover file handling. folium was implemented to display a station mapping widget, packaged with a Natural Earth scale basemap for offline use.

Library versions are listed in Table 1.

Table 1. Libraries, versions and licenses

Package	Version	Use	Licence
Python	3.12	Language	PSF-2.0
numpy	2.5.3	Matrix algebra	BSD-3-Clause
pandas	3.0.6	File export	BSD-3-Clause
matplotlib	3.11.2	Plots	matplotlib licence
seaborn	0.13.2	Heatmaps	BSD-3-Clause
PyQt6	6.11.0	Graphical user interface	GPL-3.0
PyQt6-WebEngine	6.11.0	Web view for the station map	GPL-3.0
pyqtgraph	0.14.0	Plots of large matrices	MIT
folium	0.20.0	Station map	MIT
openpyxl	3.1.5	xlsx export through pandas	MIT
plyer	2.1.0	Desktop notifications	MIT

Taken from requirements.txt and the README of release 1.5.2 (Dossas and Ampatzidis, 2026).

2.2 Installing

The following commands install the software. They copy the repository, make a virtual environment with Python 3.12 and install the dependencies of Table 1:

```
git clone https://github.com/gerxd/SINEX_TRF_Studio
cd SINEX_TRF_Studio
uv venv .venv --python 3.12
uv pip install -r requirements.txt
```

You can also use the launcher for your operating system. On Windows, double-click run_windows.bat. On macOS and Linux, run run_macos_linux.sh in a terminal. If uv is not installed, the launcher downloads it into the program folder. Then it starts main.py with Python 3.12 from uv, and main.py installs the dependencies into .venv. An internet connection is required for the first run. Later starts skip the installation.

To make a desktop shortcut to the launcher, use Settings > Create desktop shortcut (Section 5.7).

2.2.1 Installing on Windows

To install the software on Windows with the launcher, do these steps:

1. Download the source code ZIP file of the latest release from https://github.com/gerxd/SINEX_TRF_Studio/releases.
2. Extract the ZIP file to a folder near the drive root, for example C:\SINEX_TRF_Studio. The path of the folder must have fewer than 240 characters.
3. Open the folder. Double-click run_windows.bat.
4. Wait while the launcher does the setup. The setup occurs one time only. If uv is not installed, the launcher downloads uv for x86-64 or ARM64 processors into the .uv folder. It uses curl, or PowerShell if curl is not available. Then uv gets Python 3.12, and main.py installs the dependencies into .venv. When the setup is complete, the main window opens.

   NOTE: If the launcher shows “The setup did not finish”, examine the internet connection. Then double-click run_windows.bat again.

To start the program later, double-click run_windows.bat.

2.3 Starting the program

For program initialisation, run:

uv run main.py

You can also start the launcher of Section 2.2. Upon first start, main.py checks the environment and logs “The environment is ready.”. Later starts skip this check until requirements.txt changes. To examine or repair the environment, start main.py with one of these options: --check shows the state of the environment, --plan shows the installation commands, and --reinstall builds the environment again.

If the start fails on Windows with “The folder path is too long”, the path of the program folder has 240 characters or more. Move the folder nearer to the drive root. If the first start fails on Linux with “A system library is missing”, Qt needs a system library that is not installed. On Debian and Ubuntu, install the libraries that the message gives with sudo apt install.

3 Quick start with the sample data

3.1 Sample files

The features not exercised by the ITRF runs (Section 8) are shown on two small public GNSS solutions, distributed with the software. COE08827.SNX is a weekly solution of the CODE analysis centre for GPS week 882 (1996), in SINEX 1.00 from Bernese 4.1, with 37 stations and 111 position parameters (CODE, 1996). Its SOLUTION/STATISTICS block gives a variance factor of 0.002161, the same number as the RMS of unit weight (0.0022) in its header comment, so the value stored is σ_0 rather than σ_0^2. CRI23527.SNX is a loosely constrained weekly SIRGAS solution of IGN Costa Rica for GPS week 2352 (2025), in SINEX 2.01 from Bernese 5.2, with 87 stations, 261 position parameters and a variance factor of 0.5968 (IGN Costa Rica, 2025). Neither file carries velocities, so the datum effect is computed for seven parameters. Both files are public products and are redistributed with credit to their producers.

The sample data set consists of two files found at /examples.

File	Content
COE08827.SNX	CODE weekly solution, GPS week 882 (1996): SINEX 1.00, Bernese 4.1 ADDNEQ, 37 stations, 111 position parameters. SOLUTION/STATISTICS stores σ_0 (0.002161) as the variance factor. SOLUTION/MATRIX_APRIORI constrains 12 of the 111 parameters (KOSG, MADR, TROM, WTZR).
CRI23527.SNX	IGN Costa Rica weekly SIRGAS solution, GPS week 2352 (2025): SINEX 2.01, Bernese 5.2, 87 stations, 261 position parameters, loosely constrained (±1 m), variance factor 0.5968.

3.2 Walkthrough

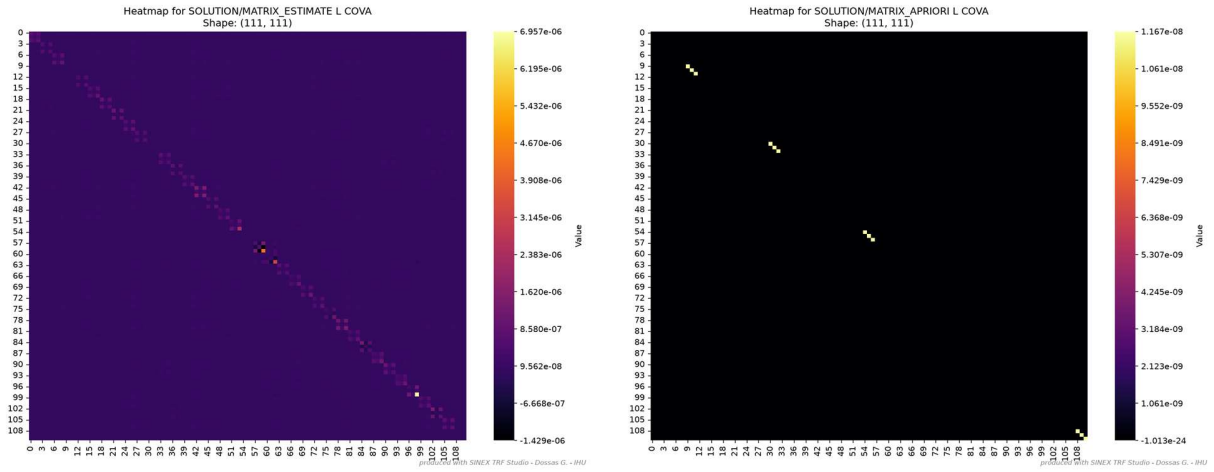
1. Load COE08827.SNX.
2. Make sure that the status widget shows the variance factor 0.002161.

   NOTE: This value is σ_0. The normal matrix needs σ_0^2.

3. Click Edit, select Custom and type 0.00000467 (0.002161^2).
4. In the Covariance Matrix tab, calculate N. The log shows the a priori block and its dimension.
5. Calculate the rank of N, then u.
6. Run the recomputation check. Check the output result is PASS.
7. Export the four matrices.
8. Plot a heatmap of each matrix and the eigenvalue distribution of the covariance matrix (Figures 1 to 3).
9. In the Datum Effect tab, calculate SigmaTheta, the cross correlations and the Helmert parameter sigmas. The default thresholds (50 mm, 3 mm/yr) exclude no episode of this file.
10. Click Filter Options, set Position to 0.001 m and click OK (Figure 8). Calculate the three products again. Ensure that the log reports 15 episodes flagged for exclusion.
11. Load CRI23527.SNX. Ensure that the status widget shows the variance factor of the file, 0.596846752908865. Do steps 4 to 6 and step 9 again.

4 Expected results

The walkthrough of Section 3.2 should give the results below. Values are given to the digits shown. A difference in the last digit can come from the numerical libraries of the platform. The values, the figures of this chapter and the screenshots in Chapters 3 to 5 come from version 1.5.2 on macOS 27 (Apple silicon), with Python 3.12.14 and NumPy 2.5.3.



(**a**) (**b**)

Figure 1. Covariance matrix heatmap of COE08827 (a), a priori covariance matrix of COE08827 (b)

Step 7 writes COE08827_Reconstructed.EXT, COE08827_Covariance_Apriori_Matrix.EXT, COE08827_NormalMatrix.EXT and COE08827_u_Vector.EXT (Section 7). The sample data set also holds the command-line runs of the same steps, with the datum products and the diagnostics reports. (Online Resource 2)

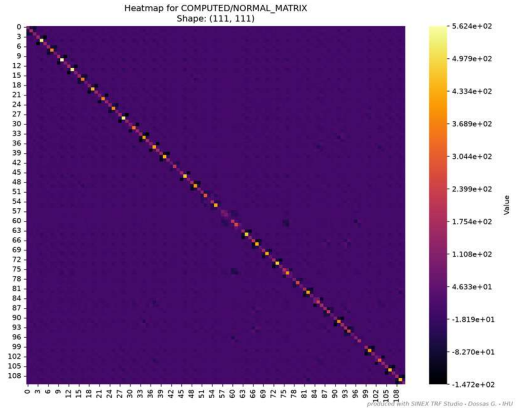


Figure 2. Normal matrix heatmap of COE08827

COE08827, variance factor 4.67 × 10^{-6}

- 111 parameters and 37 stations. The a priori block is inverted on a 12 × 12 subspace (four stations).
- Rank of N: 111 of 111. Recomputation check: relative error 1.1 × 10^{-14}, PASS.
- Covariance matrix: eigenvalues from 4.75 × 10^{-9} to 7.23 × 10^{-6} m², condition number 1.5 × 10^{3} (Figure 3).
- Datum effect without a filter: Helmert parameter sigmas tx 1.10, ty 1.55, tz 0.78, εx 1.29, εy 1.28, εz 0.88 and δs 0.37 mm. cond(R) 2567, mean |r| 0.446. The log warns that the positions have three different reference epochs.
- Datum effect filtered at 1 mm: 15 of 37 episodes excluded. Helmert parameter sigmas tx 1.12, ty 2.27, tz 1.08, εx 1.81, εy 1.47, εz 1.45 and δs 0.33 mm. cond(R) 6029, mean |r| 0.443.

CRI23527, variance factor 0.5968 from the file

- 261 parameters and 87 stations. Rank of N: 261 of 261. Recomputation check: relative error 2.1 × 10^{-13}, PASS.
- Covariance matrix: eigenvalues from 8.43 × 10^{-10} to 2.09 × 10^{-5} m², condition number 2.5 × 10^{4}.

- Datum effect without a filter: Helmert parameter sigmas tx 0.40, ty 0.49, tz 0.53, εx 0.21, εy 0.14, εz 0.17 and δs 0.16 mm. cond(R) 18.4, mean |r| 0.278.

On the regional COE08827 network, the 1 mm filter removes 15 of 37 stations, and most sigmas and cond(R) increase. With fewer stations, the network geometry determines the datum less well, so a stricter filter does not always give a better result.

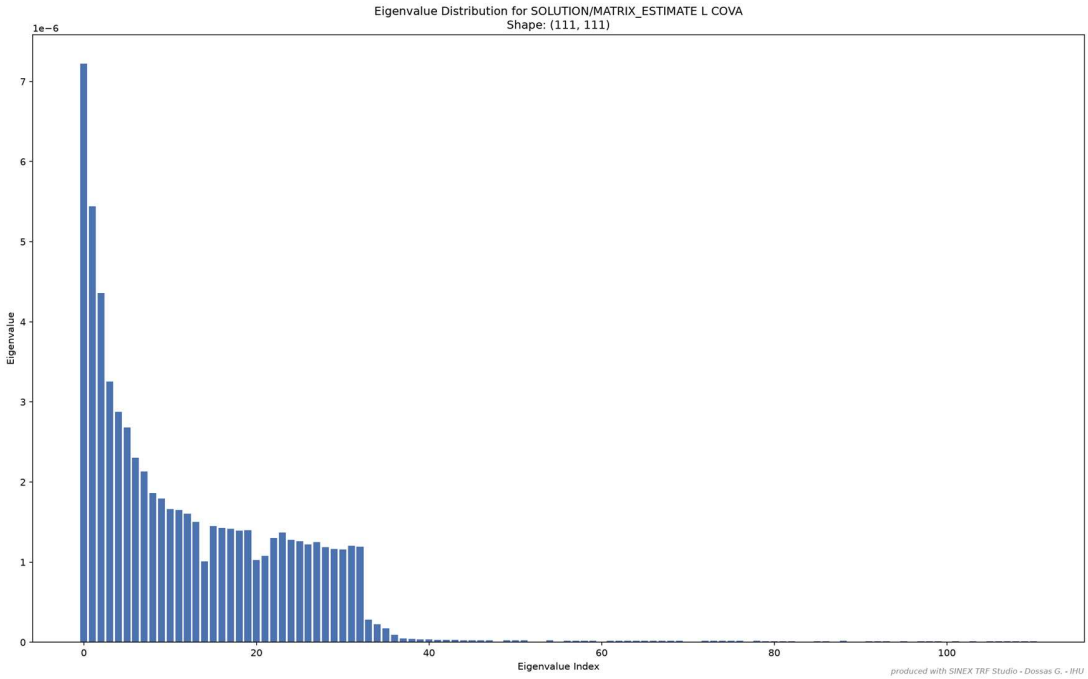


Figure 3. Eigenvalue distributions of the covariance matrix of COE08827

5 Program reference

5.1 Status widget and variance factor

The status widget is at the top left of the window. It shows data about the file that you load. The most important item is the variance factor. The widget shows if the file contains a variance factor. If the file does not contain a variance factor, you can enter a custom value. You can also enter a custom value to replace the value from the file.

5.1.1 Custom variance factor widget

To set a custom variance factor, click Edit next to "Variance Factor" in the status widget. The Variance Factor dialog opens (Figure 4).

The dialog has two options. "Default (from file)" uses the value of the SOLUTION/STATISTICS block. The dialog shows this value, or N/A if the file has no value. "Custom" uses the value that you type, from 10^{-12} to 10^{12}. Click OK to apply the option.

The status widget then shows "Variance Factor: <value> (custom)". If the file has no variance factor and you did not set a custom value, it shows "Variance Factor: Missing".

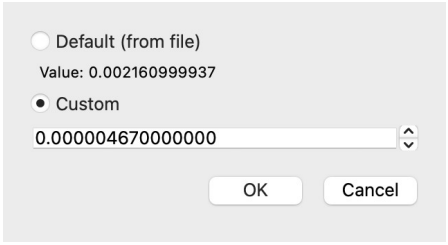


Figure 4. The Variance Factor dialog with a custom value of 4.67×10^{-6} for COE08827

The software uses the variance factor only for N and the products that follow from it (Section 5.3). The datum effect calculation does not use it. When you change the value, the software deletes the products of the Covariance Matrix tab. The custom value applies only to the loaded file. When you load a new file, the software uses the value of that file.

5.2 Benchmark recording

The software can record the time of each operation. This option is off by default. To record the times, select Settings > Record benchmark. The software uses the Python time library to measure the start and the end of each operation. It also records the peak memory, which gives an approximate memory footprint. To save the times as a text file, select Settings > Export benchmark report…. This item is available after the software has recorded times.

5.3 Covariance Matrix tab

5.3.1 Inputs

The tab uses these data from the SINEX file:

- The estimated parameter values x_{est}, from SOLUTION/ESTIMATE.
- The a priori parameter values x_{apr}, from SOLUTION/APRIORI.
- The variance factor σ_0^2, from SOLUTION/STATISTICS, or a custom value (Section 5.1).
- The covariance matrix C_x of the estimated parameters, from SOLUTION/MATRIX_ESTIMATE.
- The a priori covariance matrix C_0 of the constraints, from SOLUTION/MATRIX_APRIORI.

The software uses C_x, C_0 and the variance factor to calculate N (Section 5.3.2). It uses x_{est} and x_{apr} to calculate u and to do the recomputation check (Section 5.3.3).

5.3.2 Normal equation matrix

In the Covariance Matrix tab (Figure 5), the software calculates the normal equation matrix N from the covariance matrix and the variance factor. The file must contain both. ITRF solution files do not contain a variance factor. For these files, enter a value with the custom variance factor widget (Section 5.1). When you enter a new variance factor, the software deletes all products that it calculated with the old value. The tab also has a recomputation check (Section 5.3.3).

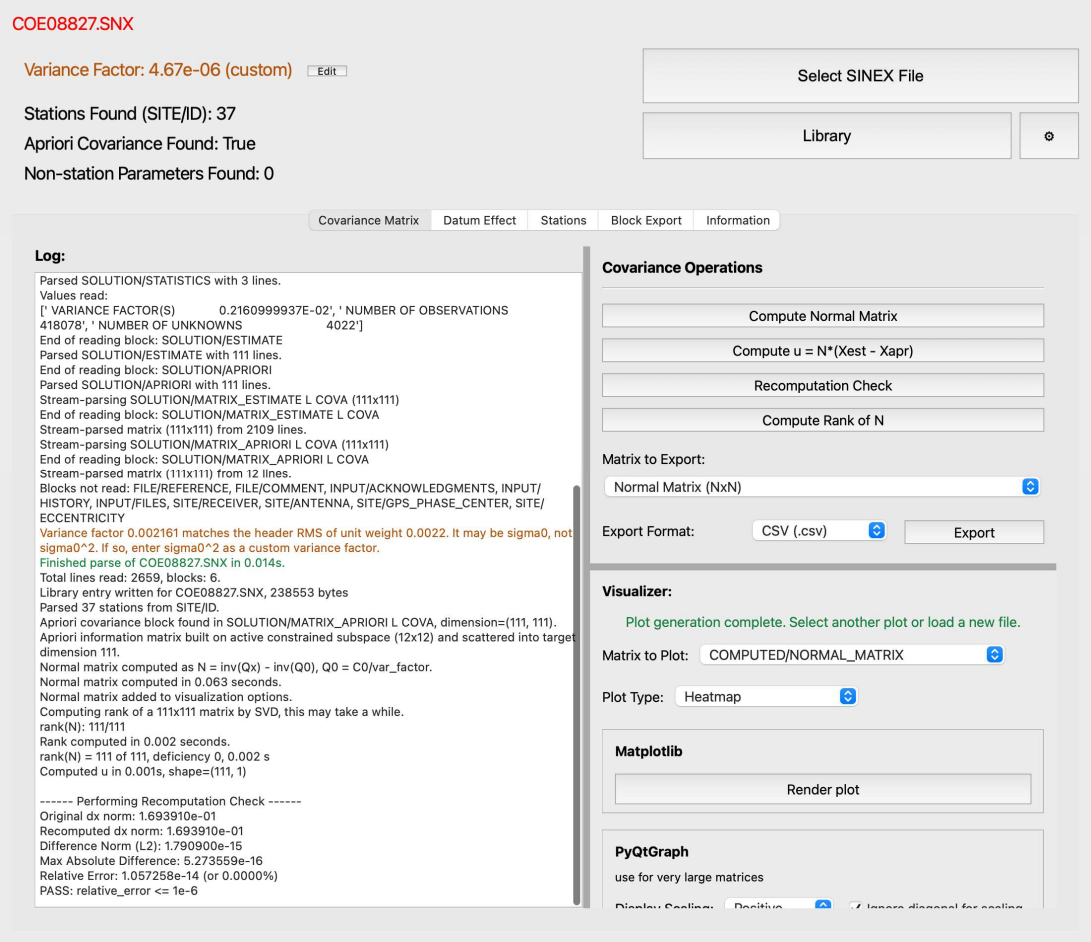


Figure 5. The Covariance Matrix tab for COE08827 after computing N, its rank, u and the recomputation check, with the log on the left and the covariance operations, export and visualiser controls on the right

A critical background process to note is the a priori constraint removal. Often the a priori covariance block is present but holds no values, so the software first checks whether the block contains data. Similarly, an additional software behaviour is the way it handles the matrix after confirming it contains data: the a priori covariance matrix frequently contains sparse values. However, if the covariance matrix is large (meaning the normal matrix is too) the a priori covariance matrix must match it in size, in order to allow for the subtraction of its inverse. Inverting a matrix with tens of thousands of empty elements just to store, for example, 10 values is a highly unoptimised solution, since it requires allocating in memory the whole matrix, even if it is mainly empty. Therefore, the active subspace is stored exclusively, inverted and then refilled with zeroes.

Consequently, the normal equation matrix is computed with the following:

$$N = Q_x^{-1} = \sigma_0^2 C_x^{-1} \quad \text{when no a priori covariance exists} \qquad (1)$$

$$N = Q_x^{-1} - Q_0^{-1} = \sigma_0^2 (C_x^{-1} - C_0^{-1}) \text{ when a priori covariance exists} \quad (2)$$

where $Q_0 = C_0/\sigma_0^2$, with C_0 the a priori covariance matrix of SOLUTION/MATRIX_APRIORI.

Finally, the rank of N is computed by singular value decomposition and relayed in the log.

5.3.3 Recomputation check

The recomputation check can be run following normal matrix reconstruction. The process involves subtracting a priori values from the estimates to form dx. Parameters present in the SOLUTION/ESTIMATE but absent from SOLUTION/APRIORI get an a priori value of 0 and a warning is logged. The software forms $u = Q_x^{-1}\, dx$ and recomputes $dx' = (N + Q_0^{-1})^{-1} u$, essentially measuring the numerical consistency of the constraint removal and the inversion.

To run the check, click the recomputation check button. The log shows the L2 norm of dx − dx′, the infinity norm and the relative error. The result is PASS if the relative error is less than 10^{-6}. A fail logs a warning.

5.3.4 Matrix exports and plots

Matrix exports are available in four formats, listed in Table 2, with inherent limitations to their floating-point accuracy.

Table 2. Matrix export options

Format name	Precision
npy	Full float64, binary[1]
txt	Full decimal precision[2]
csv	Full decimal precision[2]
xlsx	16 significant digits[3]

[1] Binary float64, written with numpy.save, so exported values are the computed ones. [2] Text, written with numpy.savetxt at 17 significant digits, which reproduces each float64 value exactly. [3] Written through pandas and openpyxl at 16 significant digits, so a value can change in its last binary digit when it is read back.

Use .npy files when possible. They have the best accuracy and are the smallest, because they are binary. The other formats are for ease of use. An .xlsx sheet holds at most 16384 columns. For a larger matrix, use .npy, .csv or .txt.

All major matrix products are exportable in the four formats of Table 2, as listed in Table 3.

Table 3. Available matrix exports

Matrix name	Origin
Reconstructed covariance matrix	SOLUTION/MATRIX_ESTIMATE L/U COVA
Normal equations matrix	Computed
A priori covariance matrix	SOLUTION/MATRIX_APRIORI L/U COVA
u vector	Computed

To plot a matrix, use the visualisation widget. It has two methods: matplotlib and PyQtGraph. Use PyQtGraph for very large matrices. It renders each cell as one pixel with GPU rendering. Table 4 gives the matrices that you can plot.

You can also plot the eigenvalue distribution of each matrix.

Table 4. Available matrix plots

Plot type	Available for matrix	Backend
Heatmap	Covariance, a priori covariance, normal	matplotlib
1:1 heatmap	Covariance, a priori covariance, normal	PyQtGraph
Eigenvalues	Covariance, a priori covariance, normal	matplotlib

5.4 Datum Effect tab

5.4.1 Datum effect computation

In the datum effect tab (Figure 6) lies the datum effect workflow options (Figure 7). This process involves constructing the matrices required for computing the covariance matrix of the frame parameters (Σ_θ), as proposed by Kotsakis and Chatzinikos (2023).

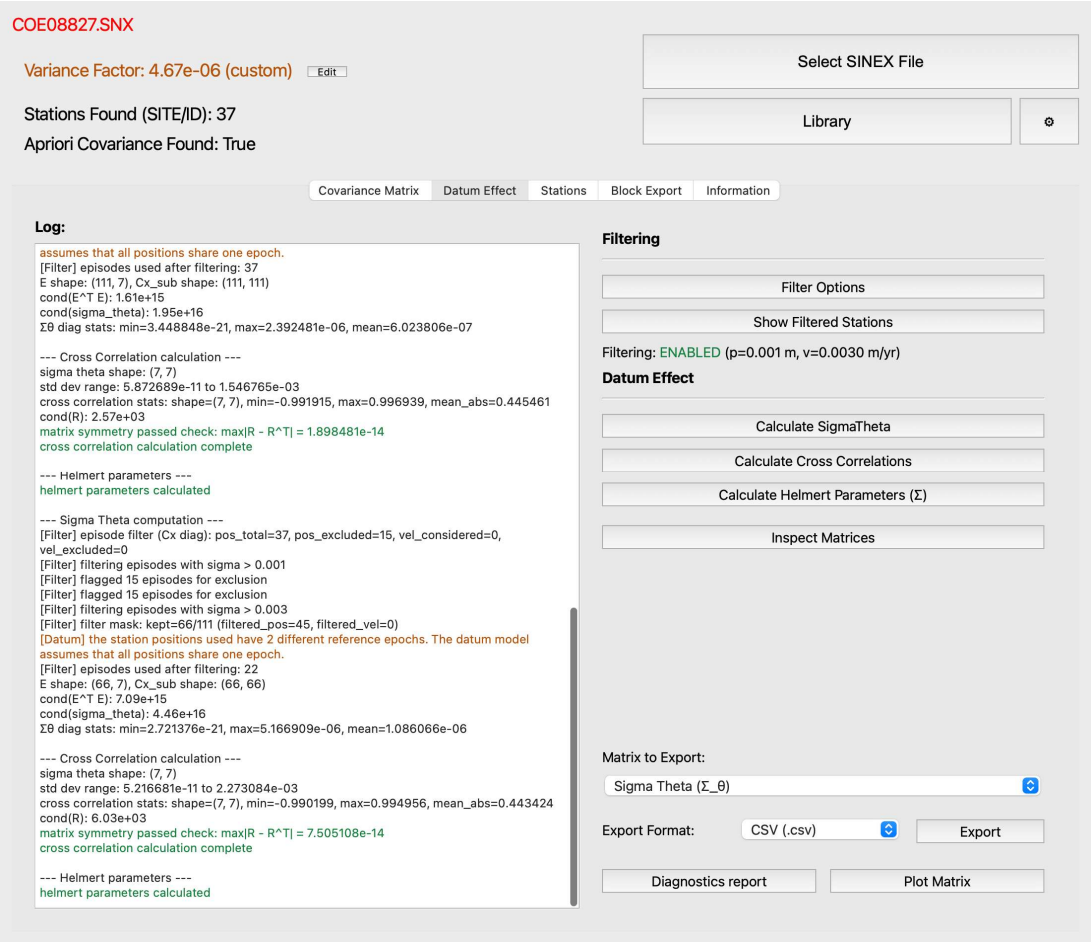


Figure 6. The Datum Effect tab for COE08827 after a run filtered at 1 mm, with the filter, Σ_θ, correlation and Helmert parameter sigma output in the log

Parameters are sourced from the SOLUTION/ESTIMATE block. If the SINEX file contains position parameters only, the matrix formed is 3×7. If velocities are also included, the matrix columns are 14.

Initially E_i is constructed, for every episode, as:

$$E_i = \begin{bmatrix} 1 & 0 & 0 & X & 0 & Z & -Y \\ 0 & 1 & 0 & Y & -Z & 0 & X \\ 0 & 0 & 1 & Z & Y & -X & 0 \end{bmatrix} \qquad (3)$$

E is then built as the stack of E_i matrices, per episode, with the following structure:

$$E = \begin{bmatrix} E_1 & 0 \\ 0 & E_1 \\ \vdots & \vdots \\ E_m & 0 \\ 0 & E_m \end{bmatrix} \qquad (4)$$

If an episode is missing a parameter value, it gets skipped.

The covariance matrix of the frame parameters is then calculated as:

$$\Sigma_\theta = G\, C_{x,\mathrm{sub}}\, G^T, \quad G = (E^T E)^{-1} E^T \qquad (5)$$

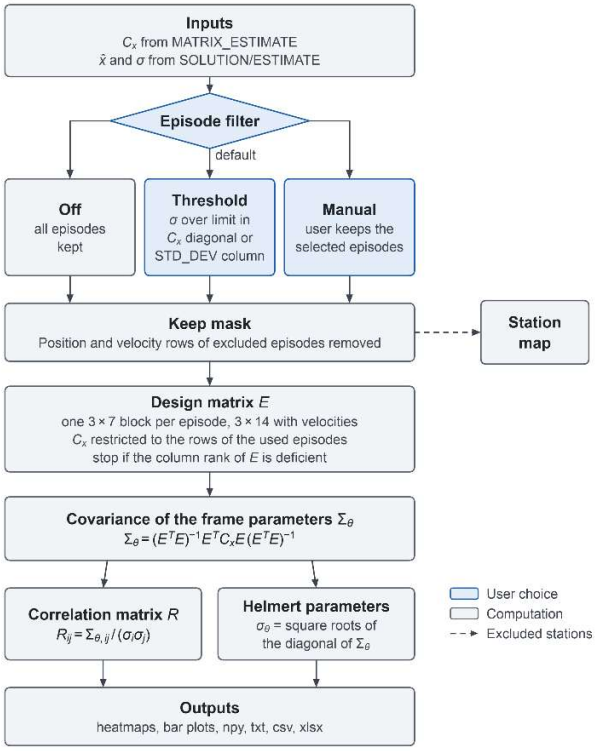


Figure 7. Flowchart of the Datum Effect tab

After its computation, several diagnostics get logged: condition numbers, and the min, max and mean of the diagonals.

The Helmert parameter sigmas can then be extracted from the covariance matrix of the frame parameters diagonals, defined as Equation (12).

The dimensions match the covariance matrix of the frame parameters. The reports give all uncertainties in mm and mm/yr: the software multiplies the rotations (radians) and the scale (unitless) by the GRS80 semi-major axis, a = 6378137 m (Moritz, 2000).

Next, the software calculates the correlation matrix R (Equation (11)). It reports the usual NumPy diagnostics: the minimum and maximum of the off-diagonal entries, the mean absolute off-diagonal entry, the condition number and a symmetry check with a tolerance of 10^{-12}. If too few episodes remain to determine the Helmert parameters, E is rank deficient, and the calculation stops with a message that gives the number of episodes, the rank and the number of parameters. The log shows a warning when the station positions used have more than one reference epoch.

One numerical caveat to note within the diagnostics, is that the condition number of Σ_θ is dominated by the mixed units of the parameters, so cond(R) is the reliable conditioning diagnostic.

5.4.2 Episode and uncertainty limit filtering

Prior to calculating the datum effect, you can apply a filter to the episodes. Set an upper threshold for the position sigma and for the velocity sigma. The software rejects each episode that has a parameter above the threshold. If all the episodes of a station are above the threshold, the software removes the station completely. Thus, the filter removes low-accuracy stations and low-accuracy episodes.

The process modifies the covariance matrix of the frame parameters computation operation, specifically the construction of the E matrix.

The filter keeps one flag for each episode. For each episode, the software compares the standard deviation of each parameter (positions, and velocities where present) with its threshold. If one parameter is above its threshold, the software rejects the whole episode. The standard deviations come from two sources, and either source can reject an episode: the STD_DEV column of SOLUTION/ESTIMATE and the square roots of the diagonal of the covariance matrix. The software then keeps only the rows and columns of Cx that belong to the retained episodes, the submatrix Cx,sub, and uses it in Equation (5).

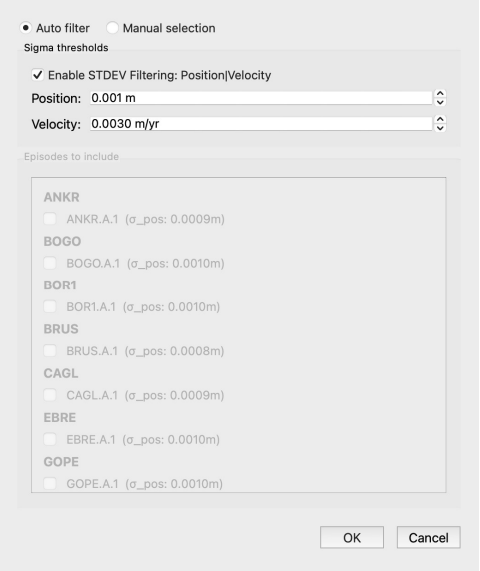


Figure 8. The Filter Options window for COE08827, with the automatic threshold filter and the manual episode selection

A manual episode selection is also available. The list shows the parameter values and the data of each episode. All filter options are in one window (Figure 8).

The code assumes that all velocity-bearing stations also contain position parameters, as well as that there are not any discrepancies with the storing following a consistent fashion. All three major matrix products are exportable in the four formats of Table 2 and can be plotted through matplotlib.

5.5 Stations tab

The Stations tab shows a map of the stations (Figure 9). The map uses a Natural Earth GeoJSON basemap, which is included with the software. The software shows the map with folium in a PyQt6 WebEngine widget. Each station is a pin on the map and an item in the list. The episodes of one site are in a small ring around it. To see the SITE/ID data of a station, click its item in the list. After a filter, a red dot shows each filtered station and a green dot shows each kept station. Use the two checkboxes to show the filtered stations, the kept stations or all stations.

The plotting works by sourcing the data from the SITE/ID block and converting the coordinate data from DMS. The SITE/ID coordinates are themselves approximate, which suits the nature of this widget, being a secondary locator widget rather than a results plotter widget.

The base map is using Natural Earth data, attributed in map_assets/ATTRIBUTION.md.

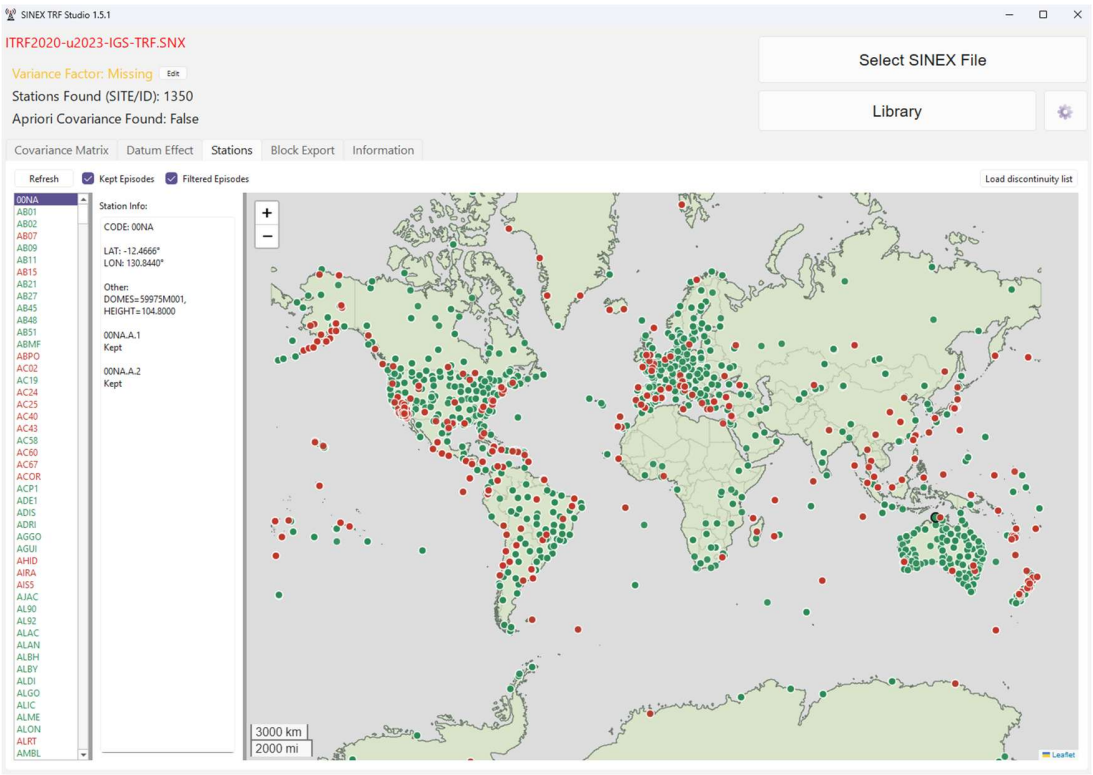


Figure 9. The Stations tab for the filtered ITRF2020-u2023 GNSS (IGS) solution, with kept episodes in green and filtered episodes in red

To load a discontinuities list, use the Stations tab. The software matches each entry to its station and episode. The info section then shows the reason for each discontinuity.

5.6 Block Export tab

The Block Export tab (Figure 10) lists each parsed block (version 1.5 and later). You can export each block alone, in the usual formats. A preview window shows the values. In the ESTIMATE and APRIORI blocks, the shading of each cell shows its sigma.

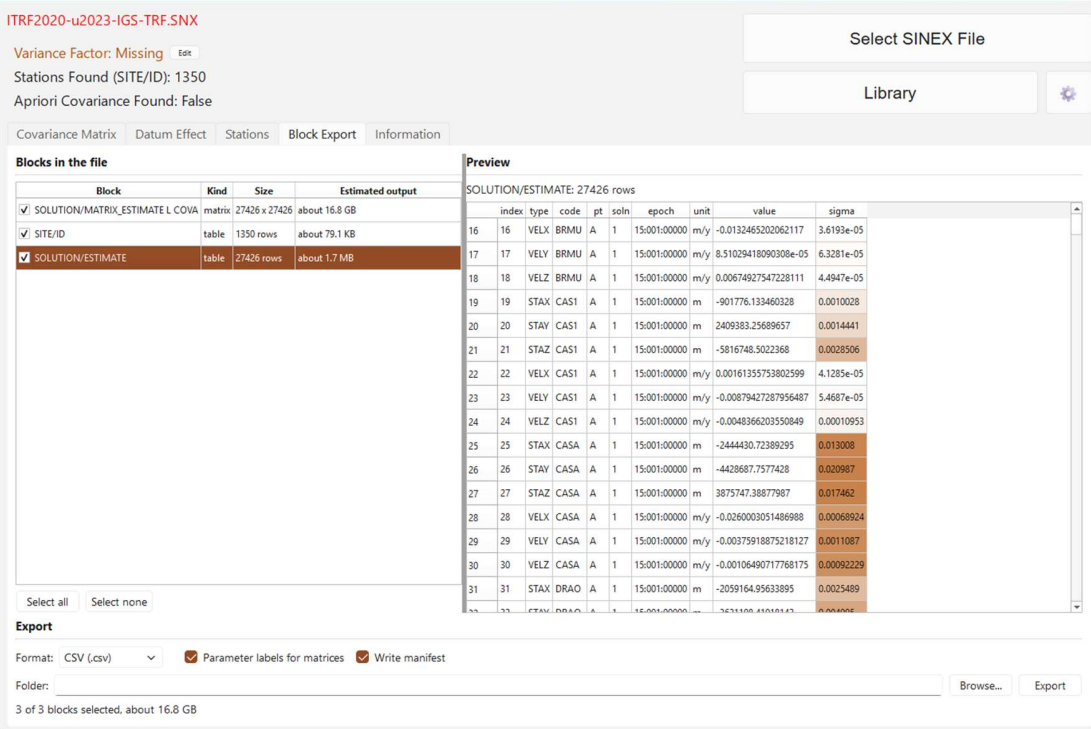


Figure 10. The Block Export tab for ITRF2020-u2023-IGS-TRF.SNX, listing the parsed blocks with their sizes, and a preview of SOLUTION/ESTIMATE with the sigmas shaded by rank

5.7 Settings menu

The software logs all operations with the Python logging tool. The log shows each action, error and success message. The console shows the same lines. Colours show the type of each action. You can also write the log to a dump file. Set the size, location and behaviour of the dump file in the Settings menu.

To open the Settings menu, click the ⚙ button to the right of Library. The software keeps the settings between sessions. Changes to the parsing options apply to the next load. (Figure 11)

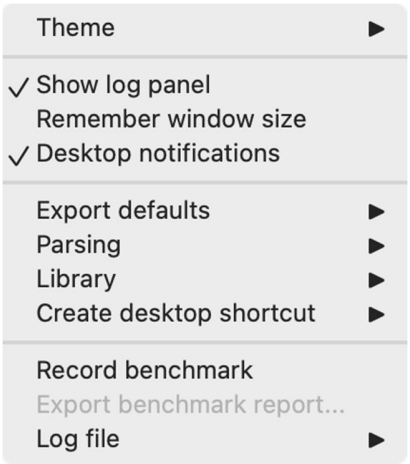


Figure 11. The Settings menu

Theme: Light, Dark or Follow system. Default: Light.

Show log panel: shows the log panel of the tabs. Default: on.

Remember window size: Default: off.

Desktop notifications: Default: on.

Export defaults > Format: NumPy (.npy), CSV (.csv), Text (.txt) or Excel (.xlsx). Default: CSV.

Export defaults > Choose folder… sets a fixed export folder. Use the last used folder removes it. Default: the last used folder.

Export defaults > Figure dpi: 100, 150, 200 or 300. Default: 150.

Export defaults > Export datum matrices without labels. Default: off.

Parsing > Skip file validation: skips the check of the block structure before the parse. Default: on.

Parsing > Skip parsing SOLUTION/EPOCHS. Default: on. The episode data spans need this block.

Parsing > Parse large files in parallel. Default: on. Parsing > Parallel workers: 2, 4 or 8. Default: 4.

Parsing > Remember filter thresholds. Default: off. When this option is off, the thresholds start at 50 mm and 3 mm/yr.

Library > Keep a binary copy of loaded files. Default: on. Library > Library folder… sets the library folder (Section 5.8).

Create desktop shortcut: makes a shortcut to run_windows.bat or run_macos_linux.sh.

Record benchmark. Default: off. Export benchmark report… saves the recorded times (Section 5.2).

Log file > Write log file. Default: on. The software writes sinex_parser.log to the folder that the program starts from, and the menu shows the path. Choose folder… sets a different folder. Size limit: No limit, 1 MB, 10 MB or 50 MB. Default: No limit. Clear log file empties the file. Console level: Warnings, Info or Debug. Default: Warnings.

5.8 Binary file library

When “Keep a binary copy of loaded files” is on (default), the software keeps a copy of each file that you load in the library. The copy holds the parsed blocks, with each matrix as a binary .npy file. When you load the same file again, the software opens the copy and reads the matrices from disk when it needs them (memory-mapped).

The library is located in the folder “library” in the program folder. If the program folder is not writable, the software uses the application data folder of the user and shows a warning in the log. To use a different folder, select Settings > Library > Library folder…. The library is then the subfolder “SINEX TRF Studio library” of that folder.

The software finds the copy of a file by its name, size and modification time. If these do not agree with an entry, it calculates the SHA-256 of the file and uses it as the name of the entry. The software parses the file again if the entry is incomplete, if a different version of the software wrote it, or if different parsing options wrote it (validation, SOLUTION/EPOCHS). Thus, entries are not used across versions. A .gz file gets decompressed to a temporary folder for the parse.

To see the library of available files, click Library. The Library window shows one row for each entry: File, Folder, Compressed, Parameters, Station episodes, Blocks, Size on disk, Source size, Added, Last opened and Version. An entry from a different version shows in the warning colour. To load an entry, select it and click Load. To delete an entry, select it and click Delete. You are barred from deleting the entry of a currently loaded file. Open folder opens the library folder. The last line gives the number of entries and the disk space that they use. (Figure 12)

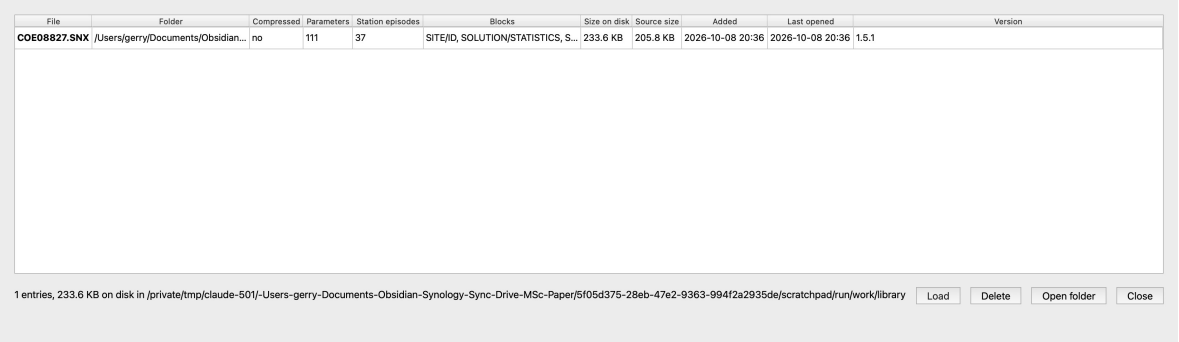


Figure 12. The Library window with one entry

Each n × n matrix uses approximately $8n^2$ bytes on disk.

5.9 Information tab

The Information tab shows the version of the software, the authors, the contact data, the dependencies and an overview of the program (Figure 13).

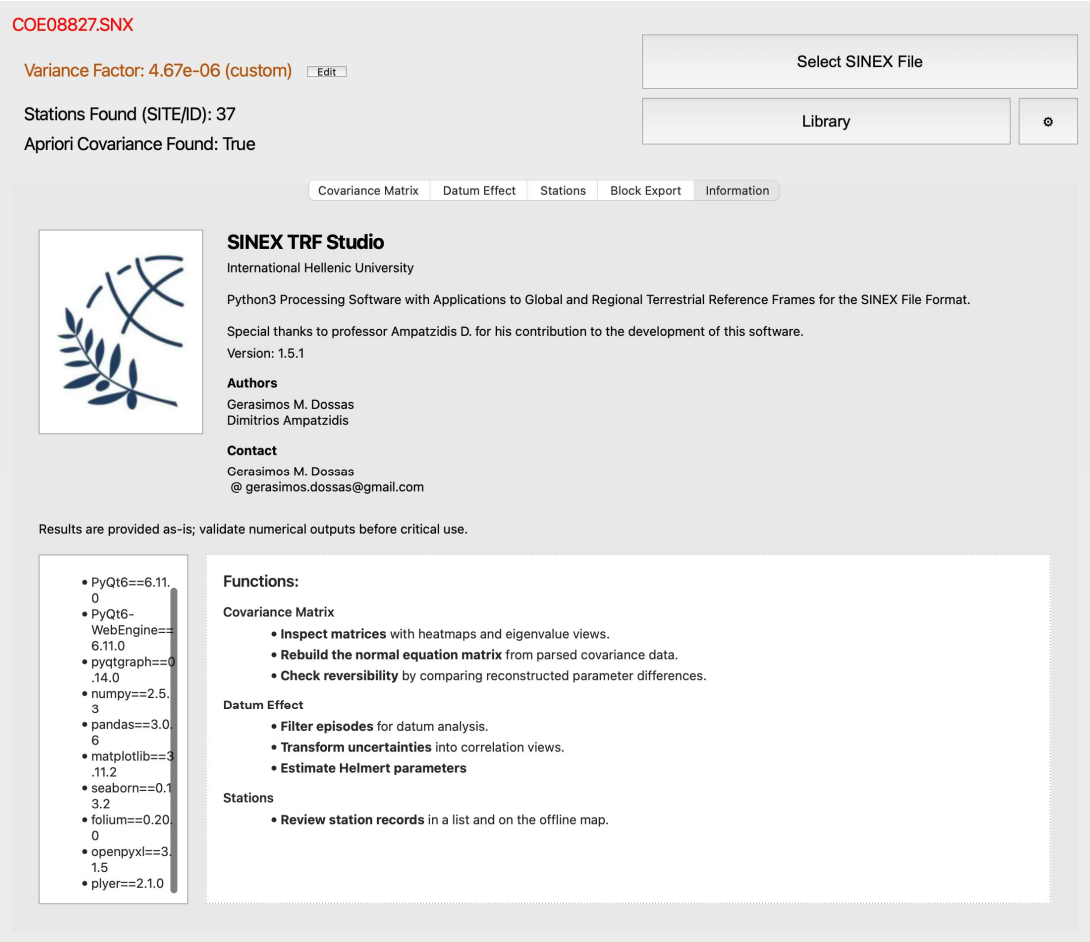


Figure 13. The Information tab

6 Command line

Headless, terminal execution can be run with:

python -m sinex_parser.cli <command> [options]

Use the Python of the virtual environment: .venv/Scripts/python.exe on Windows and .venv/bin/python on macOS and Linux.

Replace <command> with one of these four commands. parse lists the blocks of a file. datum calculates Σθ, R, the Helmert parameter sigmas and the diagnostics report, with the episode and uncertainty limit filter. report writes only the diagnostics report. normal calculates N, u and the recomputation check. For the same input and settings, the commands give the same results as the window. Example:

python -m sinex_parser.cli datum examples/CRI23527.SNX --out out --no-filter

To see the options of a command, type the command with --help. For the full reference, refer to docs/CLI_GUIDE.md in the repository. It gives the options and their default values, the exit codes, the JSON output and batch examples.

7 Output files

Each file name starts with the name of the loaded file without its extension, shown below as FILE. EXT is the export format (Table 2). The save dialog proposes the name, and you can change it.

Covariance Matrix tab

- FILE_NormalMatrix.EXT: the normal equation matrix N
- FILE_Covariance_Apriori_Matrix.EXT: the a priori covariance matrix
- FILE_u_Vector.EXT: the u vector
- FILE_Reconstructed.EXT: the covariance matrix of the estimates

Datum Effect tab

- FILE_sigma_theta[TAG].EXT: Σθ
- FILE_cross_correlations[TAG].EXT: the correlation matrix R
- FILE_helmert_parameters[TAG].EXT: the Helmert parameter sigmas
- FILE_diagnostics[TAG].txt and FILE_diagnostics[TAG].json: the diagnostics report

TAG is added after a filter: _filtered_pPOSmm_vVELmmyr for a threshold filter, or _manual_Nexcl for a manual selection of N excluded episodes. For example, ITRF2014-IGS-TRF_sigma_theta_filtered_p50mm_v50mmyr.csv. The datum matrices include parameter labels, unless “Export datum matrices without labels” is on (Section 5.7).

Command line (datum --plots)

- FILE_sigma_theta[TAG].png, FILE_cross_correlations[TAG].png and FILE_helmert_parameters[TAG].png: the figures

Block Export tab

- FILE_BLOCK.EXT: one file for each block, with each "/" and space of the block name changed to "_", for example FILE_SOLUTION_ESTIMATE.csv
- FILE_BLOCK_params.csv: the parameter labels of a matrix block
- export_manifest.json: the list of the files written

Other files

- FILE_benchmark.txt: the benchmark report (Section 5.2)
- sinex_parser.log: the log file (Section 5.7). With a size limit, the older part is kept as sinex_parser.log.1
- One library folder for each loaded file, named with its SHA-256, with entry.json, blocks.json and one .npy file for each matrix (Section 5.8)

8 Reproducing the published runs

The two global solutions are not distributed with the software because of their size. They are available from the ITRF website at the addresses below.

Reference timings for version 1.5.2, measured on an AMD Ryzen 5 5600 with 32 GB of RAM on Windows 11, from the exported benchmark files:

ITRF2020-u2023 IGS solution (9.9 GB, 27426 parameters): the covariance block was streamed and the file parsed in 107 s, Σθ was computed in 9.2 s, and the peak memory of the process was 11.9 GB. Version 1.1 needed about 360 s to read the same file.

ITRF2014 IGS solution (4.1 GB, 17616 parameters): the file was parsed in 39 s, Σθ was computed in 3.5 s, and the peak memory was 5.2 GB.

Timings depend on the processor, the disk and the available memory, and should be read as indicative. The memory columns of the benchmark file are filled on Windows only.

8.1 ITRF2014 GNSS (control run)

The ITRF2014 GNSS solution (ITRF2014-IGS-TRF.SNX; Altamimi et al., 2016), available at https://itrf.ign.fr/en/solutions/ITRF2014, was run as a control against the independent results of Kotsakis and Chatzinikos (2023), whose STDs.txt and CORREL.txt products accompany their article. The run applied their 5 cm / 5 cm/yr filter and compared the Helmert parameter sigmas and the correlation matrix R with those products. An unfiltered run was added for control.

The solution has 2936 episodes and 17616 parameters. The filter excluded two episodes, both because of their position sigma. The velocity criterion never triggered, with the largest velocity sigma in the file being 5.70 mm/yr against the 50 mm/yr threshold. After the filter, 2934 episodes and 17604 parameters remain. Since the two filters operate on the same unit at the same thresholds, the retained networks are identical. The thresholds sit at roughly 60 to 780 times the mean sigmas of the file.

The largest absolute difference from the published values is 0.0049 mm. Table 5 lists the fourteen values against the reference.

Table 5. ITRF2014 GNSS solution filtered at 5 cm / 5 cm/yr: Helmert parameter sigmas of the realised frame against the reference values, in mm and mm/yr. Parameter order of Kotsakis and Chatzinikos (2023), with the rotations and the scale multiplied by the GRS80 semi-major axis

Parameter	Unit	**Reference** (Kotsakis and Chatzinikos, 2023)	Filtered
tx	mm	0.51	0.51
ty	mm	0.48	0.48
tz	mm	0.55	0.55
εx	mm	0.16	0.16
εy	mm	0.21	0.21
εz	mm	0.31	0.31
δs	mm	0.51	0.51
tx_v	mm/yr	0.03	0.03
ty_v	mm/yr	0.03	0.03
tz_v	mm/yr	0.03	0.03
εx_v	mm/yr	0.01	0.01
εy_v	mm/yr	0.02	0.02
εz_v	mm/yr	0.02	0.02
δs_v	mm/yr	0.03	0.03

The comparison extends beyond the diagonal. R, reordered to their parameter order, has a condition number of 76.3 and a mean absolute off-diagonal correlation of 0.1311 (Table 7).

The filter removes two episodes. It does not remove a complete station. The fourteen standard deviations change by 0.23% on average, and no correlation coefficient changes by more than 0.008. The technique solutions that enter the ITRF have already passed three rounds of outlier removal, at the analysis centres, in the technique-specific combination and in the multi-technique combination of the ITRF combination centres (Zhang et al., 2024). Figure 14 shows the correlation matrix of the filtered run, and Figure 15 the fourteen Helmert parameter sigmas of the realised frame.

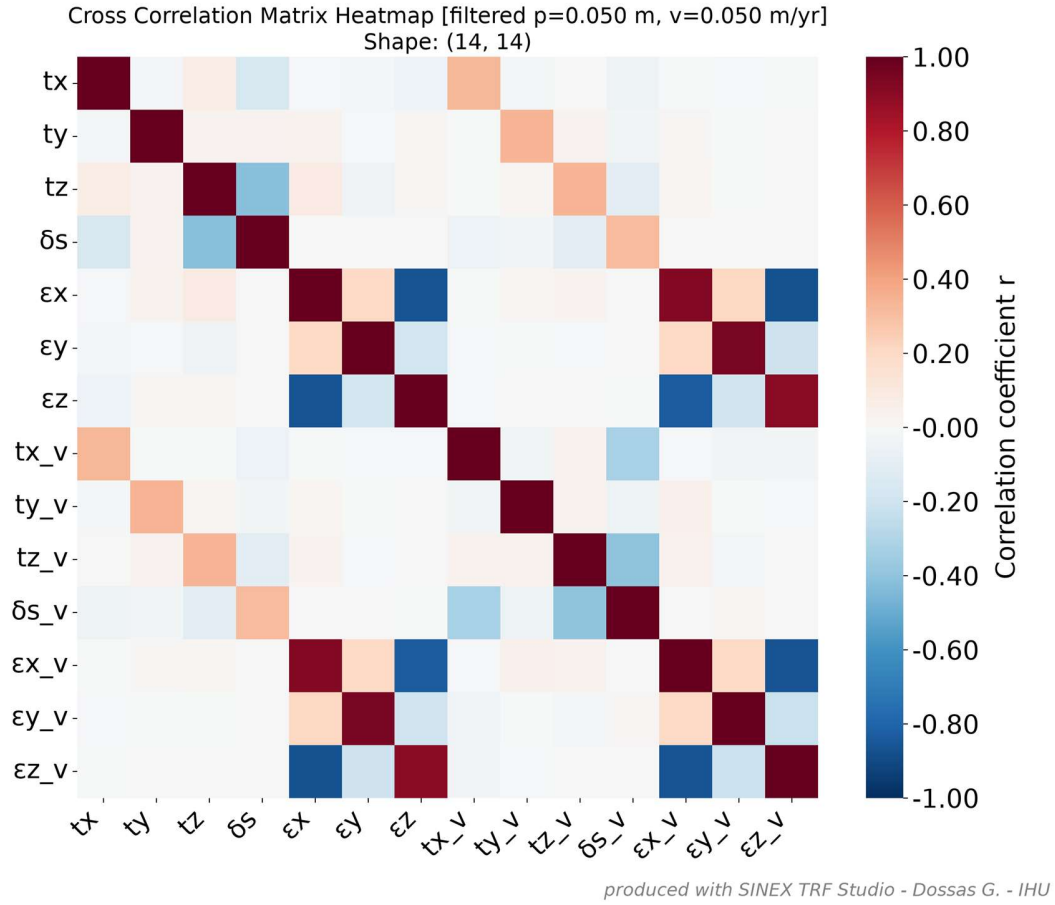


Figure 14. Correlation matrix *R* for the ITRF2014 GNSS solution filtered at 5 cm / 5 cm/yr (parameter order of the software)

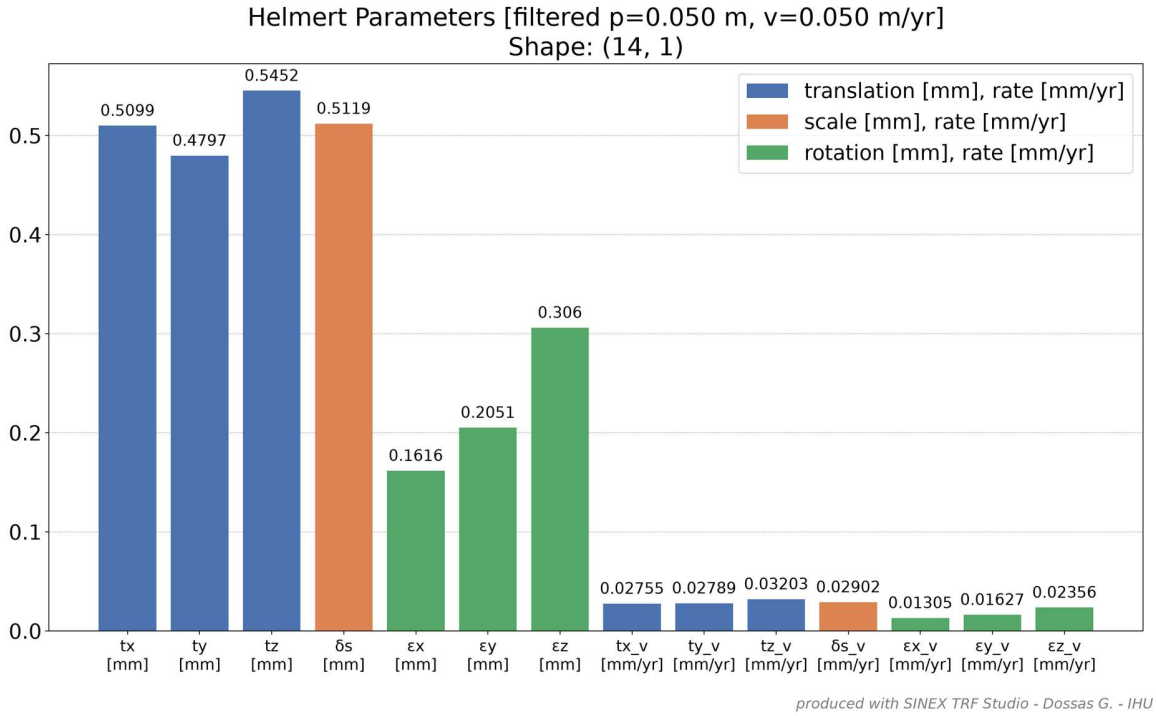


Figure 15. The fourteen Helmert parameter sigmas of the realised frame for the ITRF2014 GNSS solution after filtering at 5 cm / 5 cm/yr, in the software parameter order. Units are mm and mm/yr, with rotations and scale scaled by the GRS80 semi-major axis

8.2 ITRF2020-u2023 GNSS

A second run used the ITRF2020-u2023 IGS solution (Altamimi et al., 2024; Johnston et al., 2017), available at https://itrf.ign.fr/en/solutions/itrf2020-u2023, once unfiltered and once filtered at 1 cm / 1 mm/yr, to show the effect of the filter on the datum products.

Kotsakis and Chatzinikos (2023) cover ITRF2000 to ITRF2014 and leave ITRF2020 to a later paper because of its non-secular modelling of station motions. The filter excluded 715 of the 4571 episodes. These include all the episodes of 94 of the 1350 stations, so the filter removed these stations completely. Table 6 lists the fourteen Helmert parameter sigmas before and after filtering.

Table 6. ITRF2020-u2023 GNSS solution without a filter and filtered at 1 cm / 1 mm/yr: Helmert parameter sigmas of the realised frame, in mm and mm/yr. Parameter order of Kotsakis and Chatzinikos (2023), with the rotations and the scale multiplied by the GRS80 semi-major axis

Parameter	Unit	Unfiltered	Filtered
tx	mm	0.69	0.10
ty	mm	0.89	0.10
tz	mm	0.48	0.08
εx	mm	0.50	0.07
εy	mm	0.40	0.07
εz	mm	0.45	0.06

Parameter	Unit	Unfiltered	Filtered
δs	mm	1.07	0.14
tx_v	mm/yr	0.16	0.01
ty_v	mm/yr	0.19	0.02
tz_v	mm/yr	0.12	0.01
εx_v	mm/yr	0.11	0.01
εy_v	mm/yr	0.09	0.01
εz_v	mm/yr	0.10	0.01
δs_v	mm/yr	0.24	0.02

Every standard deviation decreases. Those of the translations, rotations and scale decrease by factors of 5.8 to 8.5 and those of their rates by 8.9 to 12.3, the rates improve more. Filtering also changes the correlations among the parameters. Table 7 gives two metrics from the diagnostics report, the condition number of R and the mean absolute off-diagonal correlation, for both runs.

Table 7. Correlation metrics of the fourteen Helmert parameters of the realised frame, before and after filtering

Run	Threshold	Retained / total	Removed	cond(R) unfiltered	cond(R) filtered	mean |r| unfiltered	mean |r| filtered
ITRF2014 GNSS	5 cm / 5 cm/yr	2934 / 2936	0.07 %	75.0	76.3	0.1310	0.1311
ITRF2020-u2023 GNSS	1 cm / 1 mm/yr	3856 / 4571	15.6 %	401.3	29.0	0.4507	0.1668

On GNSS, after filtering, the translation–rotation pairs tx with εy and ty with εx, and their rates, are the strongest couplings (0.67 to 0.76, from 0.80 to 0.85 before). Before filtering, the tz and ty rates with the scale rate were the strongest (0.86).

Appendix A: Background, mathematical modelling and equations

The SINEX File Format

SINEX is a fixed-width, block-structured ASCII format, with each block opened by "+" and closed by "-" followed by its name, and comment lines starting with "*". The complete specification is IERS (2006). Table 8 lists the blocks that the software uses.

Table 8. Block catalogue

Block name	Contents	Use case
SOLUTION/STATISTICS	A posteriori variance factor σ_0^2 and solution counts	Scaling the covariance to a cofactor matrix
SOLUTION/ESTI	Estimated parameters with epoch, unit	The parameter list which defines the

Block name	Contents	Use case
MATE	and sigma	matrix dimension
SOLUTION/APRIORI	A priori parameter values	Differencing for dx
SOLUTION/MATRIX_ESTIMATE L/U COVA	Covariance of the estimates, one triangle stored	Stochastic input to the normal-equation and datum-effect pipelines
SOLUTION/MATRIX_APRIORI L/U COVA	A priori constraint covariance	Constraint removal
SITE/ID	Station code, DOMES number, approximate coordinates	Station map

Covariance Matrix

The covariance matrix is stored in SOLUTION/MATRIX_ESTIMATE L/U COVA.

This is a symmetrical, square matrix. Only one, triangular side is saved inside the file and denoted as L for lower and U for upper triangular. Its dimension is set by the total number of estimated parameters in the solution (three position components, and where present three velocity components, for every station episode) rather than by the number of stations. The diagonal values are the variances (σ^2) of the individual estimated parameters, so their square roots are the formal standard deviations σ. The off-diagonal values are the covariances between parameters. These are not restricted to the components of a single station: parameters belonging to different stations are correlated as well, and it is this cross-station coupling that makes the datum effect computable from the covariance matrix alone (Kotsakis and Chatzinikos, 2023).

The a posteriori variance factor (σ_0^2) is a separate scalar quantity, reported in the SOLUTION/STATISTICS block, that scales the matrix as a whole. The cofactor matrix follows from it as:

$$Q_x = \sigma_0^{-2}\, C_x \qquad (6)$$

A reduction in σ_0^2, or in an individual σ, denotes higher precision, that is a tighter formal, internal uncertainty of the solution. It does not by itself denote higher accuracy which can only be established by comparison against an external reference. The formal σ values are the quantity the filtering algorithm operates on, as described in Section 5.4.2.

Normal Matrix

While not found in final solution SINEX files, the normal matrix is relevant as a product of the software. It is a symmetrical, square matrix, that can be found typically in “NEQ” SINEX files.

The matrix is based on the classical Gauss-Markov model (Dermanis and Fotiou, 1995): $N = A^T P A$ where A is the design matrix, P the weight matrix of the observations and b the vector of reduced observations.

$$N = A^T P A, \quad u = A^T P b, \quad N\hat{x} = u \qquad (7)$$

From Equations (6) and (7), $Q_x = N^{-1}$ and $N = \sigma_0^2 C_x^{-1}$. A priori constraints add Q_0^{-1} to the normal equations:

$$C_x = \sigma_0^2 (N + Q_0^{-1})^{-1} \;\Rightarrow\; N = \sigma_0^2 (C_x^{-1} - C_0^{-1}) \quad (8)$$

$$u = Q_x^{-1}(x_{\text{est}} - x_{\text{apr}}) \quad (9)$$

The Datum Effect Formulas

Covariance Matrix Σ_θ

It is a reference frame's specific Helmert parameter covariance matrix. Its dimensions depend on whether velocities (rates) are introduced in the system. Without them, the dimensions are 7 × 7 and with velocities included, the dimensions become 14 × 14 with the elements as shown in Table 9.

Table 9. The fourteen Helmert parameters

Symbol	Name	Optional
tx	Translation along X	No
ty	Translation along Y	No
tz	Translation along Z	No
δs	Scale	No
εx	Rotation about X	No
εy	Rotation about Y	No
εz	Rotation about Z	No
tx_v	Rate of tx	Yes
ty_v	Rate of ty	Yes
tz_v	Rate of tz	Yes
δs_v	Rate of scale	Yes
εx_v	Rate of εx	Yes
εy_v	Rate of εy	Yes
εz_v	Rate of εz	Yes

Following Equation (14) of Kotsakis and Chatzinikos (2023), the matrix is defined as:

$$\Sigma_\theta = (E^T E)^{-1} E^T C_x \, E (E^T E)^{-1} \qquad (10)$$

The diagonal elements represent the variance of the datum's degrees of freedom, while the off-diagonals represent the covariance between values. Consequently, the square roots of the diagonal values are their (σ) standard deviations.

It expresses the magnitude of uncertainty for the system's position, scale and orientation. It is, however, important to point out that the matrix relates to the system's internal accuracy (meaning the uncertainty derived from the covariance of the estimated coordinates), rather than describing the deviation of the actual, physical, reference frame (Kotsakis and Chatzinikos, 2023).

Correlation Matrix R

By normalising the covariance matrix of the frame parameters using standard correlation matrix calculation, we can transfer the matrix to a unitless domain, allowing us to isolate and expose the cross correlations between position, velocity and scale parameters, by making parameters with very different units comparable.

$$R(i,j) = \frac{\Sigma_\theta(i,j)}{\sigma_i \sigma_j}, \quad \sigma_i = \sqrt{\Sigma_\theta(i,i)} \qquad (11)$$

The R matrix is not an explicitly separate evaluation result but contains the same stochastic information as the covariance matrix of the frame parameters, normalised in a unitless domain.

The matrix adheres to the standard correlation matrix structure:

- $R(i,i) = 1$ in the diagonal values.
- $|R(i,j)| \approx 0$ indicates weak linear coupling between parameters (i,j). For jointly normal parameters, this corresponds to statistical independence.
- $|R(i,j)| \to 1$ indicates strong linear coupling, with the sign giving its direction.
- $R(i,j) > 0$: the two parameters vary together, an error in one being accompanied by an error of the same sense in the other.
- $R(i,j) < 0$: inverse coupling, an error in one being compensated by an error of the opposite sense in the other, so that the two are geometrically difficult to separate.

Helmert Parameter Sigmas

The Helmert parameter sigmas are extracted from the computed covariance matrix of the frame parameters, as the square root of its diagonals.

$$\sigma_\theta = \sqrt{diag(\Sigma_\theta)} \qquad (12)$$

The Definition of an Episode

In standard SINEX structure context, a station is identified by a four-character code in the CODE column together with its position/velocity (when present) parameters. However, it is possible that a SINEX file can contain multiple instances of the same CODE identifier, as it is not unique. This is because the station equipment could be altered (deliberately or due to natural phenomena) or because multiple antennas are deployed and sampled, leading to repeated writes of the same CODE. When multiple CODE instances occur, PT and SOLN columns are used to differentiate between them. As such, for the duration of this manual, the term “episode” is used to refer to a unique identity of the CODE, PT, SOLN triple, to avoid confusion. Table 10 lists the three identifiers.

Table 10. The episode identifiers of SOLUTION/ESTIMATE and their meanings

Identifier	Meaning	Example

Identifier	Meaning	Example
CODE	Station identifier	GRAZ
PT	Point code	A

Appendix B: Architecture and parsing engine

Architecture and workflow

Since version 1.3, the code has three layers. The io layer reads and writes the files, the analysis layer does all the calculations, and the interface layer contains the widgets. The window and the command line share the io and analysis layers, so both calculate the same products with the same code. Figure 16 shows this structure.

Figure 16 shows the workflow. The software parses the SINEX file on a worker thread, so the window stays responsive during the load, and stores each block in its own data structure. You then select the operations (Figure 17). Each group of operations has its own tab: the covariance matrix pipeline (Section 5.3), the datum effect pipeline (Section 5.4) and the station map (Section 5.5). Each tab has its own export controls and plot menu.

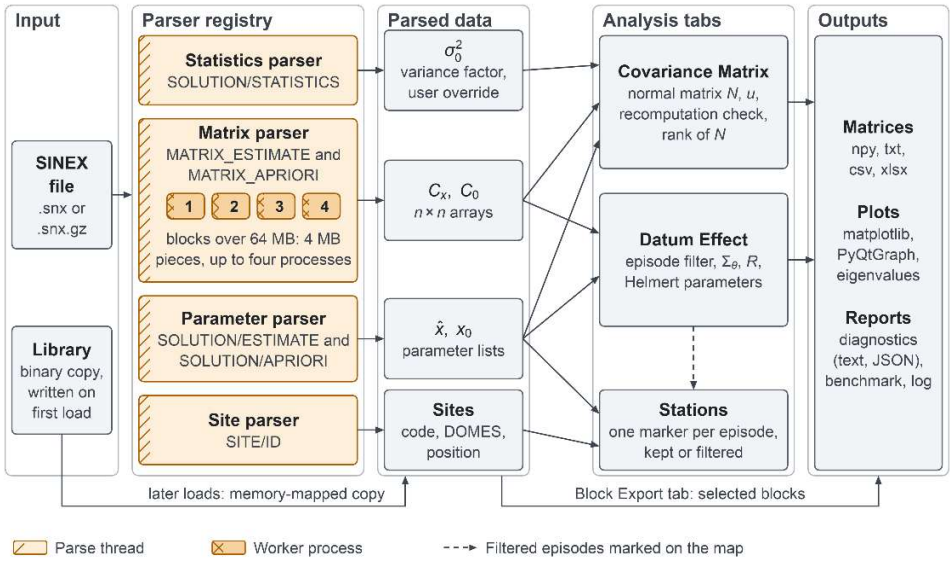


Figure 16. Typical program flow

A registry holds one parser for each block, and it can be extended: a third party can add a parser and a widget for a block that the software does not read yet. Table 11 lists the features of the software.

Table 11. Feature list

Feature	Function
Parser registry	Expandable list of parsers, each for a block
Block skipping, validation skipping	Optional SOLUTION/EPOCHS and validation skip
Asynchronous parsing	Worker thread keeps the window responsive while parsing
Tabs	Each pipeline is separated to its own tab

Feature	Function
Matrix inspector	Quick viewing of parsed and computed datum matrices
Station map	Offline map of stations
Exporting	Matrix products get exported in four formats (Table 2)
Logging	Actions, errors get written to a dump file
Benchmarking	Per-operation timings
Custom variance factor widget	Allows the overriding of the parsed σ_0^2 value
File library	Collection of SINEX files converted to binary

The software features an asynchronous parsing engine, allowing the GUI to remain responsive while loading. The main entry point of the application is main.py, that spawns a QMainWindow entity, that acts as the primary window. Each subsequent operational pipeline is sectioned off in its corresponding tab. Tabs also get their exclusive export widget, as well as their plotting menu. Matrix exports are available in four formats, listed in Table 2, with inherent limitations to their floating point accuracy.

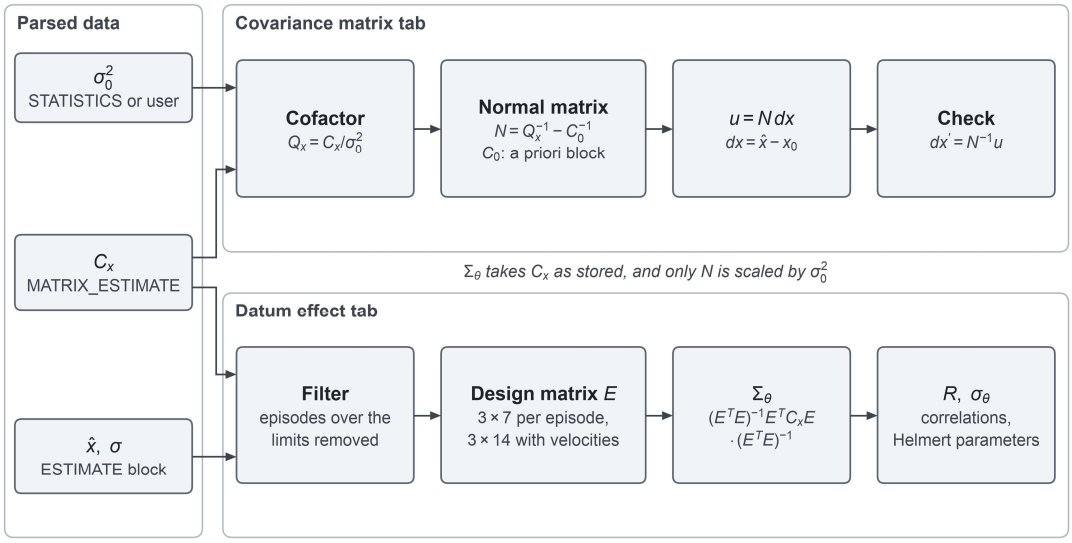


Figure 17. The two analysis pipelines that follow parsing, in the Covariance Matrix tab and the Datum Effect tab

Parsing engine

SINEX is a fixed-width format from the Fortran era, with no delimiters such as those of CSV or JSON, so standard parsing libraries do not apply. The software uses its own parsers, based on string slicing and regular expressions. Each block has a dedicated parser, and blocks with the same structure share one, as SOLUTION/ESTIMATE and SOLUTION/APRIORI do. The software loads .snx files and .snx.gz files directly.

Most SINEX files contain blocks that are unused by the software, yet occupy valuable percentage of memory. To remedy this, an optional skipping mechanic is implemented and enabled by default. In its current implementation, the function targets the SOLUTION/EPOCHS block as it multiplies the total file size, due to the fact that it contains data with similar size to the SITE/ID block while not being used for any of the software's work.

The optional validation checks the block structure against the SINEX format during the same single read of the file. If a block opens and does not close, or closes and did not open, the load stops, and a message gives the block or the line.

The covariance matrix block makes up most of the file. The software allocates the matrix once, at its final size n × n, where n is the number of SOLUTION/ESTIMATE entries (three for each episode, or six with velocities). For a block larger than 64 MiB, the software cuts the block into pieces of approximately 4 MiB at line ends. Worker processes parse the pieces into row, column and value arrays, and the main process writes the values into the matrix. The number of workers is 4 by default, and never more than the number of processor cores. Settings > Parsing > Parallel workers sets 2, 4 or 8, and "Parse large files in parallel" turns this off (Section 5.7). If the fast parse of a piece fails, the worker parses that piece line by line. A smaller block is parsed line by line in the main process. The software keeps each loaded file in the library of binary copies (Section 5.8). A later load opens the copy memory-mapped, so the matrix stays on disk and is read when an operation needs it.

The size n comes from SOLUTION/ESTIMATE, so the streamed parse needs SOLUTION/ESTIMATE before the matrix block, which is the usual order. If the matrix block comes first, the software collects the whole block and parses it at the end of the block. This is slower and needs more memory.

Appendix C: Version history

Several versions have been developed since the initial 1.1 release. The major versions can be seen in Table 12 below:

Table 12. Version history of SINEX TRF Studio

Version	Primary additions
1.1	First public release
1.2	PyQt6, Settings menu, GUI overhaul
1.3	Command line, code structure rework
1.4	Parallel covariance parsing, Block Export tab, INDEX-order and column-position reading
1.5.2	Diagnostics report, .gz input, discontinuity import, binary library
1.6 [planned]	Plug-ins API, a native plug-in for the software that fetches official SINEX files from their sources, called sinexfetch [planned]
1.7 [planned]	Stations map overhaul [planned]

Development history. Up to version 1.1.2, the development used only the usual aids of the integrated development environments (PyCharm 2024.2, Visual Studio Code 1.95): code completion and debugging. From version 1.2 to 1.5.2, AI coding agents (Claude Opus 4.8) helped with the code, the GUI, the performance tests and documentation. The authors checked and verified all the work of the agents.

References

Altamimi Z., Rebischung P., Métivier L., Collilieux X. (2016) ITRF2014: A new release of the International Terrestrial Reference Frame modeling nonlinear station motions, *Journal of Geophysical Research: Solid Earth*, Vol. 121, No. 8, 6109–6131, DOI: 10.1002/2016JB013098.

Altamimi Z., Rebischung P., Collilieux X., Métivier L., Barnéoud J., Chanard K., de La Serve M. (2024) ITRF2020-u2023, *IERS ITRS Center*, DOI: 10.18715/IPGP.2025.MH0MEM4R.

CODE (1996) Weekly EUREF solution COE08827, GPS week 0882. Center for Orbit Determination in Europe, University of Bern. EUREF Permanent Network product archive. Included in Online Resource 2.

Dermanis A., Fotiou A. (1995) Methods and Applications of Observation Adjustment (In Greek), *Ziti Publications*, Thessaloniki, Greece.

Dossas G.M., Ampatzidis D. (2026) SINEX TRF Studio, Zenodo, DOI: 10.5281/zenodo.22548394 (latest version).

Harris C.R., Millman K.J., van der Walt S.J., Gommers R., Virtanen P., Cournapeau D., Wieser E., Taylor J., Berg S., Smith N.J., et al. (2020) Array programming with NumPy, *Nature*, Vol. 585, No. 7825, 357–362, DOI: 10.1038/s41586-020-2649-2.

Hunter J.D. (2007) Matplotlib: A 2D Graphics Environment, *Computing in Science & Engineering*, Vol. 9, No. 3, 90–95, DOI: 10.1109/MCSE.2007.55.

IERS (2006) SINEX: Solution (Software/Technique) INdependent EXchange Format, Version 2.02 (1 December 2006), https://www.iers.org/IERS/EN/Organization/WorkingGroups/SINEX/SinexFormat/sinex (accessed 18 September 2026).

IGN Costa Rica (2025) Weekly loosely constrained SIRGAS solution CRI23527, GPS week 2352. SIRGAS. https://sirgas.ipgh.org. Included in Online Resource 2.

Johnston G., Riddell A., Hausler G. (2017) The International GNSS Service, *Springer Handbook of Global Navigation Satellite Systems*, eds. Teunissen P.J.G., Montenbruck O., *Springer*, Cham, Switzerland, 967–982, DOI: 10.1007/978-3-319-42928-1_33.

Kotsakis C., Chatzinikos M. (2023) Terrestrial reference frames and their internal accuracy at coordinate system level, *Journal of Geodesy*, Vol. 97, No. 11, 107, DOI: 10.1007/s00190-023-01801-6.

Moritz H. (2000) Geodetic Reference System 1980, *Journal of Geodesy*, Vol. 74, No. 1, 128–133, DOI: 10.1007/s001900050278.

Riverbank Computing (2026) PyQt6, Version 6.11.0, https://www.riverbankcomputing.com/software/pyqt/ (accessed 18 September 2026).

Zhang J., Huang C., Lian L., Zhang S. (2024) Assessment of the Improvement in Observation Precision of GNSS, SLR, VLBI, and DORIS Inputs from ITRF2014 to ITRF2020 Using TRF Stacking Methods, *Remote Sensing*, Vol. 16, No. 7, 1240, DOI: 10.3390/rs16071240.